AHB-Lite to SPI Protocol Conversion Bridge Implemented on PYNQ-Z2 Using VIVADO & Jupyter Notebook

Prof. Satendra Mane, Shitalkumar Pal, Sujal Chavan, Atharva Dhagoankar, Dhanashree Nar
Department of Electronics and Telecommunication Engineering
Vidyalankar Institute of Technology
Mumbai, India
Email: {satendra.mane, shitalkumar.pal, sujal.chavan, atharva.dhagaonkar, dhanashree.nar}@vit.edu.in

***Abstract*—The paper presents the design and implementation of an AXI4LIte to AHBLite to SPI bridge on PYNQ-Z2 board, for communication between software driven processing system (ZYNQ PS) and hardware implemented design. The projects uses python jupyter notebook interface with Memory Mapped I/O's (MMIO) to control and update hardware registers in programmable logic (PL). The implementation architecture consist of two conversion bridges - axi4lite to ahb and ahb to spi, buffer to store data - async fifo, and spi master to read and serially shift the data to spi slave at desired frequency.ahb spi bridge also does the address decoding and control part. A 26 bit clock divider variable 'ticks' is used to facilitate wide spi frequency range setting from 0.745Hz upto 50MHz. Although through physical system verification it was found that after 10Mhz the signal starts distorting and output voltage drops below 3 volts. The squareness of signals was observed till 2.5MHz beyond which signal turned triangular but output voltage was stable at 3.3 volts. So, overall operating range of the system was observed to be between 0.745Hz to 10MHz. Along with frequency and wave shaping analysis, delay analysis was also done. It was observed that after 0.5MHz delay was seen between bytes (within two 8 bits data) and packets (within two packets of multiple bytes) in case of while loop of similiar data. These two delay were due to python interpreter delay and linux memory access permission delay. The Within Byte Delay (WBD) was 15us for higher frequencies and decreases till 1MHz after which the delay is no more visible in transmission. The Within Packet Delay (WPD) was 30us for higher frequencies and decreases till 0.5MHz after which this delay also disappears. The project is limited to 1 Master 1 Slave, unidirectional, SPI 8 bit, and 0.745Hz to 50MHz SPI clock speed constraints.**

***Index Terms*—AXI4Lite, AHBLite, SPI, Protocol Conversion Bridge, PYNQ-Z2 Board, Finite State Machine (FSM), Async FIFO Buffer, Memory Mapped I/Os, Clock divider, Serial Communication, Configurable SPI speed**

I. Introduction

Modern embedded systems require efficient communication between software driven CPU and external peripherals. Protocols such as Advanced eXtensible Interface (AXI) and Advanced High Performance Bus (AHB) are widely used for on chip communication between CPU and on chip memory controllers, while Serial Peripheral Interface (SPI) is widely used for communicating to external slave devices. But, when CPU directly wants to took with external slave device, there exists a mismatch of speed and architecture between two. Hence, there arises a need for a protocol conversion bridge between high speed parallel communicating CPU following AXI or AHB and slow speed serial slave following SPI. In PYNQ Z2, the ZYNQ PS operates on AXI protocol, and in the implemented Hardware logic this AXI protocol signals are converted to AHB, then AHB to SPI for communicating to Slave at the other end. These protocol conversions require proper bridging between two protocols and careful handling of the data to avoid data corruption and data loss within the bridge and facilitate data synchronization, transaction control and timing management.

The paper presents the bridge design between AXI4Lite to AHBLite to SPI protocol, providing error free communication between software controlled processor and SPI peripheral slave. The system uses Python Jupyter Notebook interface with Memory Mapped IOs allowing the user to target and update the hardware registers and start data transmission. The wide range of frequency configuration enables both high speed operation as well as low speed debugging.

In the implemented design, multiple interconnected modules make the bridge. AXI4Lite to AHB bridge converts the incoming AXI4 address/data to AHB signals. The AHB to SPI block performs address decoding and control operations. FIFO block acts as a storage elements where high speed data can be dumped by CPU. SPI master block read this data from the FIFO buffer and shifts it serially to SPI Slave at configured SPI frequency. To facilitate variable SPI operating speed, a 26bit clock divider is used which counts till 6,71,08,864, Hence user can set a value to internal 26bit register variable clk_div through jupyter notebook ticks variable which takes 26bit value and pass on to clk_div. The ticks value and corresponding frequency formula is discussed further in the paper. This 26bit register provides frequency range between 0.745Hz to 50MHz (Theoretically).

The system was validated for correct functionality throughout frequency range, with usable operating range upto approximately 10Mhz with triangular signal output and signal voltage of 3 volts. Along with this two types of delay were observed WBD and WPD which resulted due to Python interpreter delay

and Linux Memory access delay. These tests provides system constraints for signal integrity and frequency dependent output voltage.

The brief implementation of work includes: (i) Multi protocol conversion bridge AXI to AHB, AHB to SPI, (ii) Configurable SPI clock speed, (iii) Validation of system throughout frequency range, distortion and voltage degradation analysis, (iv) Analysis for delay between bytes and packets.

II. Literature Review

Communication between heterogeneous protocols is a fundamental requirement in modern embedded and FPGA based systems. Standard on chip communication protocols such as AXI (Advanced eXtensible Interface) and AHB-Lite (Advanced High-performance Bus) are widely used for high-speed data transfer within system on chip (SoC) architectures, while Serial Peripheral Interface (SPI) is commonly employed for interfacing external peripherals due to its simplicity and efficiency. Several works have focused on the design and implementation of SPI controllers on FPGA platforms. SPI is preferred in embedded applications due to its full duplex capability, low hardware complexity, and high data rate support [1], [2].

These implementations primarily focus on standalone SPI master/slave modules without addressing integration with complex bus architectures. AHBLite protocol has been extensively studied for high performance internal communication in ARM based systems. It offers a simplified version of the full AHB protocol while maintaining parallel pipelined data transfer and reduced control overhead [3]. Prior research has explored AHB based system designs and bus arbitration techniques to improve data throughput and latency. Bridge protocol has also been an active area of research with many studies focusing on bridge conversions such as AHB to APB bridges and AXI based interconnects [4], [5]. These designs talk about techniques for data and control signal translation between different communication standards providing compatibility across system components. FIFO buffers are used in such systems to handle data transfer between modules operating at different speeds. The use of FIFO enables smooth data flow and prevents data loss [6]. Despite these advancements most existing works focus on either individual protocol implementations or simple two protocol bridges. There is limited work that integrates AXI4Lite, AHBLite, and SPI into a unified architecture with software driven control and real time hardware validation on FPGA platforms like PYNQ Z2 [7]. Therefore, this work implements a protocol bridge between AXI4Lite, AHBLite, and SPI, along with a software controlled interface using Jupyter Notebook and MMIO. The proposed system analyzes practical validation, configurable clock generation, and detailed performance analysis, distinguishing it from existing implementations.

III. Methodology

The Bridge was implemented on PYNQ-Z2 board with jupyter notebook. Jupyter notebook serves as a AHB Master, the bit file generated on VIVADO provides the required bridge interface consisting of AHB Slave to Async FIFO to SPI Master modules. The output is taken from the PMOD JA connectors on PYNQ-Z2 and observed on DSO acting as spi slave. This output is tested for speed, data correctness and python interpreter and linux memory access delay.

The 2 vivado project was created for this project, IP Creator and IP User. IP Creator project consists of all 5 design files., top_pynq.v, axi4lite_to_ahb.v, ahb_spi_bridge, async_fifo and spi_master. These files were used to create and package an IP implementation of Hardware. The Implemented Hardware base address was set to 0x4000_0000 and Range was set to 0x1000(4K) while packaging. In IP User project, the packaged IP was connected with zynq processing system in block design and Hardware address was validated in Address Editor inside block design. This block design was wrapped in HDL wrapper, and along with .xdc file final bit stream was generated. This bit stream was loaded to PYNQ-Z2 board through program device via USB port.

The current implementation of the bridge is uni-directional i.e. jupyter notebook sends the data and it is displayed on DSO. Slave device cannot send data back to the Master. The bridge is made only for 1 master - 1 slave architecture.

The command that the user (master) wants the slave to perform is written in Python using the Jupyter Notebook interface. The Python code is interpreted by the Python interpreter, which invokes the MMIO library functions to access hardware-mapped addresses. The MMIO provides actual hardware mapped address by adding the offset to the base address (For e.g., if base is 0x40000000 and offset is 0x08 then MMIO mapped address will be 0x40000008).

After this request is passed on to the Linux operating system, which then verifies whether the user has permission to access physical hardware memory through mechanisms such as /dev/mem. If permission is denied, the operation is blocked. Otherwise, Linux allows the CPU to execute the memory write instruction.

The CPU executes the write operation to the provided address, and based on the system memory map, this automatically generates an AXI4-Lite transaction between CPU and the programmable logic (PL). Inside the PL, the AXI-to-AHB bridge converts the AXI transaction into an AHB-Lite transaction.

The next block is AHB-SPI bridge it decodes the address and supports two types of write operations: control writes and data writes. Control writes updates the configuration registers such as spi_dc, clk_div, and reset, which are implemented using flip-flops in the FPGA fabric. These registers are pre-synthesized and physically mapped at the time of bit stream load from vivado through program device.

Data write occurs when the decoded address corresponds to the FIFO input register (address 0x00). In this case, the AHB-SPI bridge generates the necessary control signals to write data into the FIFO. The FIFO is a dedicated module implemented using an array of registers, created during bitstream generation, and acts as a buffer to store data bytes.

The SPI master module reads 8-bit data from the FIFO, loads it into an internal shift register, and transmits it serially based on a clock generated using the programmable clock divider (clk_div). The SPI communication follows CPOL = 0 and CPHA = 0 (Mode 0), where data is transmitted bit-by-bit over MOSI and synchronized with the SPI clock.

Finally, the SPI signals are output through the PMOD JA interface and can be observed using an SPI slave device or a digital storage oscilloscope (DSO).

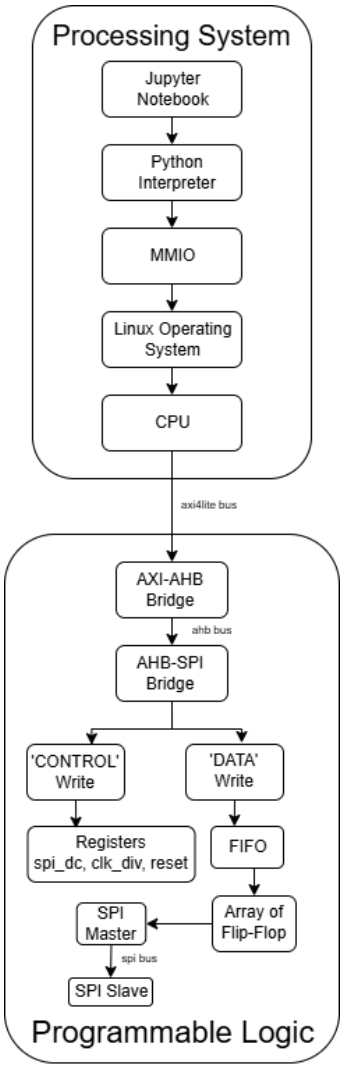

Fig. 1. System Flow

IV. Memory-Mapped Interface

The system exposes a set of memory-mapped locations categorized into 1 data port (FIFO Input), 3 control registers and 3 status registers, they support configuration, data transmission and real time status check while communication.

The data port is mapped to address 0x00 and is used to write 8-bit data into the FIFO buffer for SPI transmission. Each write operation pushes data directly into the FIFO memory. This mechanism enables continuous streaming of data from the processor to the SPI master.

TABLE I
AHBLite-SPI Bridge Register Map - Base address 0x4000_0000

Offset	Address	Type	Name	Purpose
0x00	0x40000000	Data	FIFO Data	Push data into FIFO
0x08	0x40000008	Control	spi_dc	1 = Data, 0 = Command
0x18	0x40000018	Control	slave_reset	Active low reset
0x1C	0x4000001C	Control	clk_div	SPI clock frequency
0x0C	0x4000000C	Status	FIFO Status	{dc(1 or 0, fifo_full(1 or 0), fifo_empty(1 or 0)}
0x10	0x40000010	Status	tx_count	Bytes transmitted
0x14	0x40000014	Status	FSM State	1 = IDLE, 2 = POP, 3 = LAUNCH, 4 = WAIT

The control registers are used to configure the behavior of the SPI interface and external device. These registers are implemented using flip-flops in the FPGA fabric and retain their values until updated.

(a) spi_dc Register (0x08): The spi_dc register controls whether the transmitted data is interpreted as command or data by the SPI slave device. A value of 0 indicates command mode, while a value of 1 indicates data mode. This signal is directly forwarded to the external SPI device and does not alter the SPI transmission logic internally.

(b) slave_reset Register (0x18): The slave_reset register controls the reset signal of the external SPI device. A value of 0 asserts the reset (active-low), while a value of 1 releases the device from reset, enabling normal operation.

(c) Clock Divider Register (0x1C): The clock divider(clk_div) register stores a 26-bit user provided value that determines the SPI clock frequency. The SPI clock is derived from the system clock using this programmable divider, allowing flexible control over the communication speed.

The system provides multiple status registers for monitoring internal operation. Each register returns a 32-bit value, with specific bits representing meaningful system information.

FIFO Status Register (0x0C): The status register returns a 32-bit value where only the least significant three bits are used: bit[2] represents the spi_dc signal, bit[1] indicates FIFO full condition, and bit[0] indicates FIFO empty condition, while all higher bits are zero.

Transmission Count Register (0x10): The transmission count register returns a 32-bit value representing the total number of bytes successfully transmitted by the SPI master. This counter increments each time a byte transfer is completed.

FSM State Register (0x14): The FSM state register returns a 32-bit value where only the least significant two bits represent the current state of the SPI controller. The states are encoded as IDLE = 0, POP = 1, LAUNCH = 2, and WAIT = 3, while rest all higher bits are zero.

V. Block Design

The Jupyter Notebook interface acts as the primary software interface to control and interact with the hardware system. It enables the user to send commands through Python, which are processed via the Python interpreter and translated into

low level memory access operations using the MMIO interface. The MMIO layer maps user provided address offsets to physical hardware addresses, while the Linux operating system ensures controlled access to these memory regions. The CPU then executes the corresponding memory transactions, which are forwarded to the programmable logic through the AXI4Lite interface.

This block performs two operations: data transfer and system configuration. Data transfer is achieved by writing at address 0x00, which acts as a FIFO data entry port. Each write operation pushes 8bits of data into the FIFO buffer facilitating continuous streaming of data to the SPI master. System configuration is done using control register writes, where specific addresses correspond to specific control signals. Writing to address 0x08 configures the spi_dc signal for selecting between command and data modes, while writing to address 0x18 controls the slave reset signal to reset the external SPI device.

Dynamic SPI clock control can be done by writing to 26-bit value to address 0x1C, the user configures the clock divider used in the SPI master, allowing operation over a wide frequency range (0.745Hz to 50MHz). This feature provides both high speed communication and low speed operation for debugging.

Along with control and data operations, this block also supports system monitoring through readback functionality. The user can get internal system states by reading specific memory mapped addresses including FIFO status (full/empty), transmission count, FSM state, and clock divider value. This capability facilitates real time debugging, validation, and performance analysis of the SPI communication system. The AXI4Lite to AHB bridge block is implemented as a protocol conversion block that converts the AXI4 signals to AHB signals and feeding this converted signal to AHB to SPI bridge in programmable logic. Since the CPU generates AXI4Lite transactions in PYNQ Z2 while the rest of the Programmable Logic operates on the AHBLite protocol this block is responsible for translating AXI transactions into equivalent AHB transactions.

The bridge between PS and PL accepts AXI4Lite write and read requests from the processor where address and data channels may arrive independently. To handle this the design does internal buffering using dedicated registers to store the write address, write data, and read address until a complete transaction is formed. This dedicated storage ensures correct handling of AXI handshaking signals and prevents data loss due to asynchronous arrival of address and data.

For write operations, the bridge takes the write address and write data separately and starts an AHB write transaction once both are available. It generates the corresponding AHB signals, including HADDR, HWRITE, HWDATA, and HTRANS (set to NONSEQ), to perform a single transfer. The bridge then waits for the HREADY signal from the AHB slave to confirm completion and then generates the AXI write response (BVALID and BRESP), validating successful execution to the processor.

For operations the bridge takes the read address and starts an AHB read transaction by sending the right control signals. When it gets data from the AHB slave, which it knows is valid because of HREADY it sends the data to the AXI interface through RDATA and confirms its done by asserting RVALID.

There's an internal manager, a finite state machine or FSM that oversees the steps needed for both read and write operations. This includes capturing the address carrying out the transaction and handling the response. This FSM makes sure that AXI and AHB work together smoothly for communication.

This block is specifically used for AXI4Lite single beat transactions, since no burst transfers are required in the current system. By performing reliable protocol translation and transaction management, the AXI4Lite to AHB bridge serves as a critical link between the software-controlled processing system and the hardware programmable logic.

TABLE II
SIGNAL MAPPING BETWEEN AXI AND AHB-LITE INTERFACES

AXI	**AHB**	**Description**	**Design Function**
AWADDR	HADDR	Write address	Target register/FIFO location
AWVALID	—	Address valid	Used for capturing address in buffer
WDATA	HWDATA	Write data	Data for FIFO or control registers
WVALID	—	Data valid	Used for capturing data in buffer
WSTRB	—	Byte enable	Not used (32-bit assumed)
—	HWRITE	Write control	Set to 1 for write operation
—	HTRANS	Transfer type	Set to NONSEQ (2'b10)
—	HSIZE	Transfer size	Fixed at 32-bit transfer
—	HREADY	Ready signal	Indicates AHB completion
—	HRESP	Response	Always OKAY
BVALID	—	Resp. valid	Indicates write completion to CPU
BRESP	—	Write resp.	Always OKAY response

TABLE III
READ SIGNAL MAPPING BETWEEN AXI AND AHB-LITE INTERFACES

AXI	AHB	Description	Design Function
ARADDR	HADDR	Read address	Specifies register or status to read
ARVALID	—	Address valid	Indicates valid read request
—	HWRITE	Read control	Set to 0 for read operation
—	HTRANS	Transfer type	NONSEQ to initiate read
—	HREADY	Ready signal	Indicates data is ready from AHB slave
RDATA	HRDATA	Read data	Data returned from AHB-SPI bridge
RVALID	—	Read valid	Indicates valid data available to CPU
RRESP	—	Read response	Always OKAY response

The AHB-SPI bridge is implemented as the core control and data management block of the system, responsible for interfacing the AHB-Lite bus with the SPI transmission subsystem. It performs address decoding, register control, FIFO data management, and transmission sequencing through an internal finite state machine (FSM).

The block receives AHB address, data, and control signals from the AXI-to-AHB bridge. The address is decoded using combinational logic to determine whether the operation corresponds to a control register access or a data write operation. When the decoded address corresponds to the data port (0x00), the FIFO write enable signal is asserted, allowing the incoming data to be written into FIFO memory. The FIFO internally manages the write pointer (wptr) and stores the data sequentially.

For control operations, specific address locations are mapped to dedicated registers. Writing to address 0x08 updates the spi_dc control signal, enabling selection between command and data modes for the SPI slave device. Writing to address 0x18 controls the reset signal of the external device, allowing initialization and recovery. Writing to address 0x1C updates a 26-bit clock divider register which is used by the SPI master to generate the required SPI clock frequency. These control registers are implemented using flip flops inside the programmable logic and hold their values until updated.

The block also does status monitoring through read operations. When a read transaction is started, the decoded address selects the appropriate status output. The FIFO status register provides information about FIFO full and empty conditions along with the spi_dc state. The transmission count register gives the number of successfully transmitted bytes while the FSM state register gives the current state of the transmission controller. These readback features enable real time monitoring and debugging of the system.

The internal FSM, which controls the data flow between the FIFO and the SPI master. The FSM operates through four states: IDLE, POP, LAUNCH, and WAIT. In the IDLE state, the FSM checks the fifo_empty signal to detect the availability of data. When data is present, it transitions to the POP state, where it asserts the FIFO read enable signal, causing the FIFO to output one byte of data on fifo_dout and increment the read pointer. The FSM then moves to the LAUNCH state, where it asserts the tx_start signal to initiate SPI transmission. Since fifo_dout is directly connected to the SPI master input (tx_data), the data is fetched into the SPI master's internal shift register. The FSM then enters the WAIT state, where it waits for the tx_done signal from the SPI master, validating completion of transmission. Upon completion, the FSM returns to the IDLE state and repeats the process for rest of the data if available in the FIFO.

The AHB-SPI bridge also manages flow control through the HREADY signal, which is de-asserted when the FIFO is full to prevent further writes from the processor. This ensures safe operation without data overflow. By integrating address decoding, control register management, FIFO interfacing, and FSM-based transmission control, the AHB-SPI bridge serves as the central coordination unit of the SPI communication system.

TABLE IV
AHB INPUT TO INTERNAL SIGNAL MAPPING

AHB Input	Internal Signal	Purpose
HADDR	addr	Address decoding
HWRITE	ahb_write	Enable write operation
HWDATA	fifo_din / regs	Data input to FIFO or registers
—	fifo_wr_en	FIFO write control logic
—	fifo_rd_en	FIFO read control logic
—	tx_start	SPI transmission trigger

The FIFO block is implemented as a buffering mechanism between the AHB SPI bridge and the SPI master to decouple data write and read operations. It provides smooth data flow by allowing the processor to write data at high speed while the SPI master reads and transmits data at a user defined speed. This design ensures reliable data transfer without loss or overflow under varying operating speeds.

The FIFO is designed as a synchronous FIFO in the current implementation, operating on a single clock domain (HCLK). It consists of an internal memory array used to store data bytes, along with two pointers: a write pointer (wptr) and a read pointer (rptr). The write pointer points the location where incoming data is stored, while the read pointer points the location of data to be transmitted. Both pointers are updated sequentially, enabling first in first out behavior.

Data is written into the FIFO when the write enable signal (wr_en) is asserted and the FIFO is not full. On each valid write operation the input data (din) is stored in the memory array at the location pointed by the write pointer, and the write pointer is incremented. Similarly, data is read from the FIFO

when the read enable signal (rd_en) is asserted and the FIFO is not empty. During a read operation, the data at the location pointed by the read pointer is output on dout, and the read pointer is incremented on the clock edge.

The FIFO generates status signals to indicate its current state. The fifo_empty signal is asserted when the write pointer and read pointer are equal, indicating that there is no data available for transmission. The fifo_full signal is asserted when the write pointer meets up to the read pointer in a circular manner, saying that no additional data can be written to fifo without overwriting current data. These status signals are used by the AHB SPI bridge to control write operations and manage data flow safely without loss.

The FIFO operates as a circular buffer where the memory is reused once the end of the buffer is reached. This is achieved by allowing the pointers to wrap around, ensuring proper utilization of memory. The pointer comparison logic checks correct detection of full and empty conditions without ambiguity.

By acting as an intermediate storage element the FIFO isolates the timing differences between the high speed processor writes and the slow speed SPI transmission process allowing asynchronous data flow at the system level. This design improves system performance, prevents data loss, and ensures continuous SPI data transmission.

TABLE V
FIFO INTERFACE SIGNAL DEFINITIONS

Signal	Type	Description
din	Input	Data input to FIFO (8-bit)
wr_en	Input	Enables write operation
rd_en	Input	Enables read operation
dout	Output	Data output from FIFO
full	Output	Indicates FIFO is full
empty	Output	Indicates FIFO is empty
clk	Input	System clock
resetn	Input	Active-low asynchronous reset

The SPI master block is implemented as the transmission unit responsible for converting parallel data into serial output for communication with an external SPI slave device. It receives 8bit parallel data from the FIFO through the AHB SPI bridge and transmits it bit by bit over the SPI interface based on the desired clock frequency set by the user.

The module accepts input data through the tx_data signal and starts transmission when the tx_start signal is asserted. To ensure correct operation, the design includes an internal control signal (active) that indicates whether a transmission is currently in progress. A new transmission is started only when tx_start is asserted and the SPI master is idle (active = 0), preventing data corruption due to overlapping transactions.

SPI master has a programmable clock generation mechanism. The module includes a clock divider register (clk_div), which determines the SPI clock frequency relative to the system clock. An internal counter (div_cnt) generates a clock enable signal (clk_en) whenever the divider count is reached. This clock enable signal controls the toggling of the SPI clock (spi_sclk), allowing precise control over the SPI transmission speed across a wide frequency ranges.

The data transmission is performed using an internal shift register (shreg). When a transmission begins the 8 bit input data is loaded into the shift register and the most significant bit (MSB) is placed on the MOSI line. With each clock enable event, the shift register shifts left, and the next bit is transmitted. This process continues until all 8 bits are transmitted serially to the slave.

The SPI control signals are generated following the standard SPI operation. The chip select signal (spi_cs) is made low at the start of transmission and made high upon completion. The SPI clock (spi_sclk) toggles during transmission, and data is placed on the MOSI line in synchronization with the clock edges. The current implementation follows CPOL = 0 and CPHA = 0 where data is driven on one clock edge and sampled on the opposite edge.

A bit counter (bitcnt) is used to track the number of bits transmitted. Once all 8 bits have been shifted out, the SPI master makes the active signal low, releases the chip select signal, and asserts the tx_done signal to indicate completion of the transmission. This signal is used by the AHB SPI bridge FSM to initiate the next data transfer if available.

By integrating programmable clock control shift register based serialization, and transaction management, the SPI master block provides a configurable and efficient mechanism for serial data communication with external devices.

TABLE VI
SPI MASTER MODULE SIGNAL DEFINITIONS

Signal	Type	Description
tx_data	Input	8-bit parallel data to be transmitted
tx_start	Input	Start transmission trigger
clk_div	Input	26-bit clock divider value
spi_sclk	Output	Serial SPI clock signal
spi_mosi	Output	Serial data output (MOSI)
spi_cs	Output	Active-low chip select signal
tx_done	Output	Transmission complete flag

VI. Implementation and Results Discussion

1) AHB Clock Validation

```
from pynq import Overlay, MMIO, Clocks
ol = Overlay("/home/xilinx/jupyter_notebooks/ahb_spi_9dec/system_wrapper.bit")
ol.download()
spi = MMIO(0x40000000, 0x1000)
print("FCLK0 =", Clocks.fclk0_mhz)
Jupyter Output: FCLK0 = 100
```

Fig. 2. AHB Clock Validation via Jupyter Notebook Interface

The AHB clock was validated through software level observation using the Jupyter Notebook interface. Since the AHB clock is internally driven by the system clock (100 MHz) of the processing system its correctness was verified indirectly through register access, consistent data transactions, and correct timing behavior was seen during SPI operations.

```
from pynq import Overlay, MMIO
ol = Overlay("/home/Xilinx/jupyter_notebook/project_folder/bit_file.bit)
spi = MMIO(0X40000000, 0X1000)
#Configure SPI Speed
ticks = 1 to 67,108,864
funit = ticks - 1
spi.write(0x1C, funit)
```

Fig. 3. Jupyter notebook ticks

All memory mapped read and write operations executed through MMIO were completed without timing violations or synchronization issues confirming that the AHB bus operates reliably at 100 MHz. This validation ensures that the AXI to AHB bridge and subsequent hardware modules are functioning at the intended system frequency.

2) SPI Clock Generation and Analysis

The SPI clock is generated using a programmable clock divider implemented inside the SPI master block. A 26bit register is used to store the divider value providing fine control over the SPI clock frequency throughout wide range.

An internal counter (div_cnt) increments on every system clock cycle (100 MHz). When the counter reaches the programmed divider value (clk_div), it resets and generates a clock enable pulse (clk_en). This pulse is used to toggle the SPI clock (spi_sclk).

$$f_{SPI} = \frac{f_{ahb\_clk}}{2 \times \text{ticks}} \tag{1}$$

Where:

- $f_{ahb\_clk} = 100$ MHz
- ticks $=$ clk_div $+ 1$

Each SPI clock cycle consists of one rising and one falling edge. Since the clock toggles once per divider match i.e. div_count is equal to ticks value given by the users. Two of such clock toggles are required to complete one full SPI clock cycle. Hence, the frequency is divided by 2×ticks.

The choice of a 26bit divider is critical to achieve ultra low frequency operation.

$$f_{min} = \frac{100 \text{ MHz}}{2 \times 67,108,864} \approx 0.745 \text{ Hz}$$

$$f_{max} = \frac{100 \text{ MHz}}{2 \times 1} = 50 \text{ MHz}$$

This enables human observable SPI signals (1 Hz), which is extremely useful for debugging and experimenting. At the same time, the minimum tick value that is 1 allows operation up to 50MHz, providing a very wide dynamic range of frequency value.

TABLE VII
SPI FREQUENCY TO TICK VALUE MAPPING

Target Frequency (freq)	Tick Value (ticks)
1 Hz	50,000,000
10 Hz	5,000,000
100 Hz	500,000
1 kHz	50,000
10 kHz	5,000
1 MHz	50
10 MHz	5
50 MHz	1

3) SPI Clock Validation

SPI Clock was configured using ticks variable in Jupyter Notebook for the desired SPI Speed. The correctness of the SPI clock speed was validated through spi_clk output pin on PMOD JA connector of PYNQ-Z2 board and it was observed on DSO.

At low frequencies (e.g., 1 Hz to 1 MHz), the SPI clock shows a clean square waveform with voltage levels close to 3.3 V, indicating proper digital switching behavior. As the frequency increases, the clock and data waveform shape changes from square to triangular.

```
from pynq import Overlay, MMIO
ol = Overlay("/home/Xilinx/jupyter_notebook/project_folder/bit_file.bit)
spi = MMIO(0X40000000, 0X1000)
#Configure SPI Speed
ticks = 50000000 #1Hz
funit = ticks - 1
spi.write(0x1C, funit)
```

Fig. 4. Jupyter Notebook for 1Hz

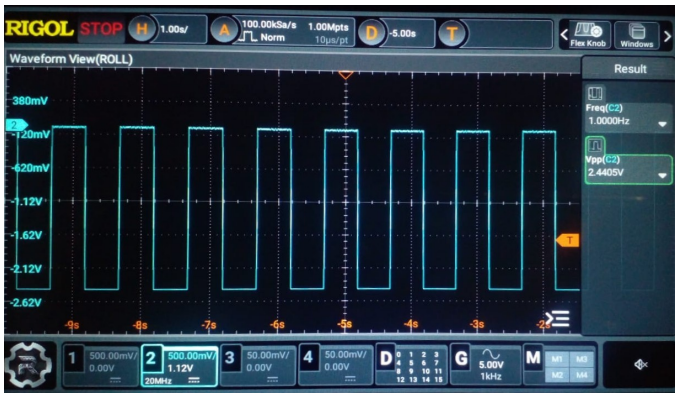

Fig. 5. 1Hz Clock on DSO

```
from pynq import Overlay, MMIO
ol = Overlay("/home/Xilinx/jupyter_notebook/project_folder/bit_file.bit)
spi = MMIO(0X40000000, 0X1000)
#Configure SPI Speed
ticks = 500000 #100Hz
funit = ticks - 1
spi.write(0x1C, funit)
```

Fig. 6. Jupyter Notebook for 100Hz

Fig. 7. 100Hz Clock on DSO

```
from pynq import Overlay, MMIO
ol = Overlay("/home/Xilinx/jupyter_notebook/project_folder/bit_file.bit")
spi = MMIO(0X40000000, 0X1000)
#Configure SPI Speed
ticks = 50000 #1KHz
funit = ticks - 1
spi.write(0x1C, funit)
```

Fig. 8. Jupyter Notebook for 1KHz

Fig. 9. 1KHz Clock on DSO

```
from pynq import Overlay, MMIO
ol = Overlay("/home/Xilinx/jupyter_notebook/project_folder/bit_file.bit")
spi = MMIO(0X40000000, 0X1000)
#Configure SPI Speed
ticks = 5000 #10KHz
funit = ticks - 1
spi.write(0x1C, funit)
```

Fig. 10. Jupyter Notebook for 10KHz

Fig. 11. 10KHz Clock on DSO

```
from pynq import Overlay, MMIO
ol = Overlay("/home/Xilinx/jupyter_notebook/project_folder/bit_file.bit")
spi = MMIO(0X40000000, 0X1000)
#Configure SPI Speed
ticks = 50 #1MHz
funit = ticks - 1
spi.write(0x1C, funit)
```

Fig. 12. Jupyter Notebook for 1MHz

Fig. 13. 1MHz Clock on DSO

```
from pynq import Overlay, MMIO
ol = Overlay("/home/Xilinx/jupyter_notebook/project_folder/bit_file.bit")
spi = MMIO(0X40000000, 0X1000)
#Configure SPI Speed
ticks = 20 #2.5MHz
funit = ticks - 1
spi.write(0x1C, funit)
```

Fig. 14. Jupyter Notebook for 2.5MHz

Fig. 15. 2.5MHz Clock on DSO

Beyond approximately 2.5 MHz, the waveform begins to appear more triangular due to signal integrity limitations such as rise/fall time constraints and hardware bandwidth. Along with shape, the voltage amplitude decreases as frequency increases. Up to around 6.25 MHz, the voltage remains within the range of 3.4 V to 3.2 V, but beyond this, a noticeable drop in amplitude is seen.

```
from pynq import Overlay, MMIO
ol = Overlay("/home/Xilinx/jupyter_notebook/project_folder/bit_file.bit)
spi = MMIO(0X40000000, 0X1000)
#Configure SPI Speed
ticks = 10 #5MHz
funit = ticks - 1
spi.write(0x1C, funit)
```

Fig. 16. Jupyter Notebook for 5MHhz

Fig. 17. 5MHhz Clock on DSO

```
from pynq import Overlay, MMIO
ol = Overlay("/home/Xilinx/jupyter_notebook/project_folder/bit_file.bit)
spi = MMIO(0X40000000, 0X1000)
#Configure SPI Speed
ticks = 8 #6.25MHz
funit = ticks - 1
spi.write(0x1C, funit)
```

Fig. 18. Jupyter Notebook for 6.25MHz

Fig. 19. 6.25MHz Clock on DSO

```
from pynq import Overlay, MMIO
ol = Overlay("/home/Xilinx/jupyter_notebook/project_folder/bit_file.bit)
spi = MMIO(0X40000000, 0X1000)
#Configure SPI Speed
ticks = 5 #10MHz
funit = ticks - 1
spi.write(0x1C, funit)
```

Fig. 20. Jupyter Notebook for 10MHz

Fig. 21. 10MHz Clock on DSO

```
from pynq import Overlay, MMIO
ol = Overlay("/home/Xilinx/jupyter_notebook/project_folder/bit_file.bit)
spi = MMIO(0X40000000, 0X1000)
#Configure SPI Speed
ticks = 4 #12.5MHz
funit = ticks - 1
spi.write(0x1C, funit)
```

Fig. 22. Jupyter Notebook for 12.5MHz

Fig. 23. 12.5MHz Clock on DSO

```
from pynq import Overlay, MMIO
ol = Overlay("/home/Xilinx/jupyter_notebook/project_folder/bit_file.bit")
spi = MMIO(0X40000000, 0X1000)
#Configure SPI Speed
ticks = 3 #16.67MHz
funit = ticks - 1
spi.write(0x1C, funit)
```

Fig. 24. Jupyter Notebook for 16.67MHz

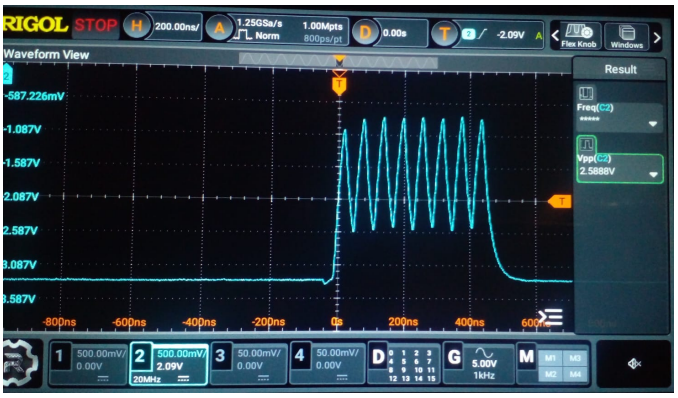

Fig. 25. 16.67MHz Clock on DSO

```
from pynq import Overlay, MMIO
ol = Overlay("/home/Xilinx/jupyter_notebook/project_folder/bit_file.bit")
spi = MMIO(0X40000000, 0X1000)
#Configure SPI Speed
ticks = 2 #25MHz
funit = ticks - 1
spi.write(0x1C, funit)
```

Fig. 26. Jupyter Notebook for 25MHz

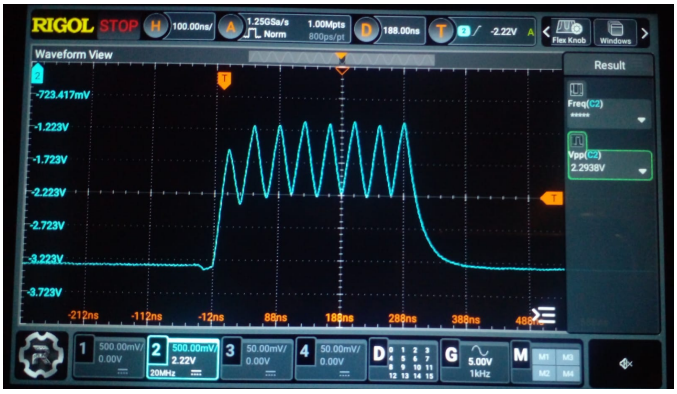

Fig. 27. 25MHz Clock on DSO

```
from pynq import Overlay, MMIO
ol = Overlay("/home/Xilinx/jupyter_notebook/project_folder/bit_file.bit")
spi = MMIO(0X40000000, 0X1000)
#Configure SPI Speed
ticks = 1 #50MHz
funit = ticks - 1
spi.write(0x1C, funit)
```

Fig. 28. Jupyter Notebook for 50MHz

Fig. 29. 50MHz Clock on DSO

At higher frequencies (above 10 MHz), the waveform becomes heavily distorted and too attenuated, making it difficult for the DSO to reliably detect the signal and measure the frequency.

TABLE VIII
SPI OPERATIONAL FREQUENCY RANGE

Parameter	**Frequency Range**
Theoretical Range	0.745 Hz – 50 MHz
Practical Range	0.745 Hz – 10 MHz
Square Wave Region	0.745 Hz – 2.5 MHz
Stable Voltage Region	0.745 Hz – 6.25 MHz

4) SPI Data Transmission Validation

SPI data transmission was validated on DSO by sending byte sequences (0xAA, 0x55, 0x0F, 0xA5, 0x3C) from the Jupyter Notebook interface in a while loop.

```
from pynq import Overlay, MMIO
ol = Overlay("/home/Xilinx/jupyter_notebook/project_folder/bit_file.bit)
spi = MMIO(0X40000000, 0X1000)
#Configure SPI Speed
ticks = 8 #6.25MHz
funit = ticks - 1
spi.write(0x1C, funit)

while True:
            spi.write(0x08,1) #Data mode
            spi.write(0x00,0xAA) #10101010
```

Fig. 30. Jupyter Notebook for data transmission at 6.25MHz

Fig. 31. Data transmission at 6.25MHz on DSO

For e.g., the jupyter notebook sends write signal to address 0x00 with data 0xAA i.e. 10101010. This signal is successfully validated in the DSO output, yellow trace is clock at 6.25MHz, blue trace is data 0xAA, and violet trace is spi_cs which is 0 while transmitting, confirming correct serial shifting of data bits over the MOSI line at different SPI frequencies.

5) Delay Analysis

Two types of delays were observed during data transmission:

Within Packet Delay (WPD): Delay between consecutive packets. Meaning if 10101010 and 11110000 was transmitted but not once but in a while loop then between each packet(10101010 & 11110000) some delay was observed based on the SPI Transmission frequency.

Within Byte Delay (WBD): Delay between consecutive bytes within a packet. Meaning if 10101010 and 11110000 was transmitted byte after byte then based on the SPI frequency of transmission some delay was observed between 10101010 and 11110000.

Fig. 32. WBD at 1MHz, 8us

Fig. 33. WBD at 2.5MHz, 12.5us

For e.g. at 1MHz frequency delay(WBD) of nearly 12.5us was seen and at 2.5MHz delay(WBD) of 8us was seen.

6) Delay Interpretation

TABLE IX
OBSERVED DELAY DATA (WPD AND WBD) ACROSS FREQUENCIES

Frequency	**WPD (μs)**	**WBD (μs)**
50 MHz	31.6	∼15.5 (variable)
25 MHz	32.1	∼15.5 (variable)
16.67 MHz	31.7	∼15 (variable)
12.5 MHz	31.7	15.08
10 MHz	30	15
8.33 MHz	30	15
7.14 MHz	30	15
6.25 MHz	30	13.6
5 MHz	30	14.2
2.5 MHz	28	12.06
1 MHz	23.9	7.58
500 kHz	14.25	0
250 kHz ↓	0	0

At higher frequencies, a nearly constant within-packet delay (nearly 30 µs) and variable within-byte delay (nearly 15 µs) are observed. These delays are attributed to software and system-level overheads, primarily due to Python interpreter execution and Linux-based memory access latency.

As the frequency decreases, these delays reduce significantly, becoming negligible below 250 kHz. This indicates that

at lower data rates, the system is no longer delayed by software overhead like interpretetion and Linux memory access delay and operates closer to ideal hardware timing.

The system supports a wide theoretical frequency range (0.745 Hz – 50 MHz) Practical operation is stable up to 10 MHz due to hardware limitations Waveform squareness quality degrades with increasing frequency Software delays are visible at higher frequencies At lower frequencies, the system behaves nearly ideal

VII. Conclusion

The implemented AXI to AHB to SPI bridge shows efficient operation for a single master single-slave architecture with unidirectional data transfer i.e. from AHB Master to AHB Slave only. The system is highly suitable for applications requiring low speed data visualization for circuit debugging, while also supporting high speed communication to be used as bridge between Master and salve in different projects. The design successfully validates both functional correctness and SPI speed configuration across a wide frequency range. The project also describes the theoretical (50MHz) and practical operating frequency range. The paper also talks about the signal shape degradation as frequency increases and distortion of the waveform. The output voltage level of the signal decreases below 3 volts after 10MHz. The square waveform was observed till 2.5MHz. Stable operating voltage range till 6.25MHz (3.4 volts to 3.2 volts). Along with these, Within Byte Delay (WBD) and Within Packet Delay (WPD) was observed at higher frequencies beyond 250KHz. These delay was introduced due to Python Interpreter delay and Linux Memory access delay. Overall, the paper covers all areas of analysis for bridge implementation on PYNQ Z2 Board.

Acknowledgment

We would like to express our sincere gratitude to our guide Prof. Satendra Mane for his valuable guidance and support throughout the course of this project. His insights and mentorship played a vital role in the successful completion of the project work. We would also like to thank our Electronics and Telecommunication Department for providing the necessary resources, including the PYNQ Z2 board, DSO, and laboratory infrastructure, because of which the practical implementation and testing of the system was possible.

References

[1] *SPI Block Guide*, Motorola Inc., 2003.
[2] J. Axelson, "Serial peripheral interface (spi)," in *Embedded Systems Handbook*, 2nd ed., 2007.
[3] *AMBA 3 AHB-Lite Protocol Specification*, ARM Ltd., 2010, aRM IHI 0033A.
[4] S. Pal and R. Kumar, "Design and implementation of spi master controller on fpga," *International Journal of Engineering Research and Technology (IJERT)*, vol. 9, no. 5, 2021.
[5] A. Kumar and P. Sharma, "Design of amba based ahb to apb bridge," in *IEEE International Conference on Communication Systems*, 2019.
[6] C. E. Cummings, "Simulation and synthesis techniques for asynchronous fifo design," *SNUG San Jose*, 2002.
[7] *PYNQ-Z2 Reference Manual*, Xilinx Inc., 2019.